

Analizzatore statico basato sull'interpretazione astratta

Università degli studi di Verona - Dipartimento
di Informatica

Alessio Terraroli

Anno accademico 2025/2026

Indice

Introduzione	2
Basi teoriche	3
Analisi statica	3
Interpretazione Astratta	3
Sintassi e semantica	4
Controllo dei tipi e validazione statica	5
Indecidibilità e Halting Problem	6
Affidabilità e precisione	7
Modellazione dei Programmi e Livelli di Analisi	8
Least Upper Bound (LUB, $\sqcup$)	9
Fixpoint Computation e Widening	9
Weak Updates	10
Implementazione	11
Specifiche	11
Guida rapida	13
Requisiti	14
Compilazione del progetto	14
Eeguire un programma e Test Bench	15
Sintassi	16
Interpretazione Astratta	16
Soundness e Approssimazione	16
LUB e Convergenza (Fixpoint)	17
Il Dominio dei Segni (Sign Lattice)	18
Struttura del Reticolo Esteso	18
Operazioni Aritmetiche Astratte	18
Componenti dell'inteprete	20

Introduzione

Il presente progetto, sviluppato nell'ambito di un tirocinio universitario, si focalizza sullo studio teorico e sull'implementazione pratica di tecniche di analisi statica e interpretazione astratta. L'obiettivo fondamentale dell'analisi statica è la verifica automatica: definire e provare in modo formale una specifica proprietà riguardante i possibili comportamenti di un programma complesso. A differenza del testing tradizionale, un approccio dinamico che si limita a osservare un insieme finito di esecuzioni e che, in caso di fallimento, produce semplicemente un'istanza scorretta rispetto alla proprietà di interesse, l'analisi statica mira a fornire garanzie formali, operando in modo automatico, sicuro (*sound*) e completo per ogni possibile esecuzione del codice. Per raggiungere tale livello di affidabilità senza incorrere in tempi di computazione infiniti, il ragionamento analitico non viene condotto sui valori concreti assunti a runtime, bensì su un'astrazione matematica dei comportamenti del software, concentrandosi esclusivamente sugli elementi strettamente legati alla proprietà che si intende verificare.

Basi teoriche

Analisi statica

L'analisi statica è una tecnica utilizzata per esaminare il codice sorgente di un programma senza eseguirlo. Questa tecnica consente di identificare potenziali errori, vulnerabilità di sicurezza, e problemi di stile nel codice. L'analisi statica può essere eseguita manualmente da sviluppatori esperti o automaticamente utilizzando strumenti software dedicati. Gli strumenti di analisi statica possono essere configurati per rilevare una vasta gamma di problemi, tra cui errori di sintassi, problemi di sicurezza, e violazioni delle convenzioni di codifica. L'analisi statica è una parte importante del processo di sviluppo del software, poiché aiuta a migliorare la qualità del codice e a ridurre il numero di bug e vulnerabilità che possono essere introdotti durante lo sviluppo.

Un interprete è un programma che esegue direttamente le istruzioni scritte in un linguaggio di programmazione, senza la necessità di compilare il codice in un formato eseguibile. L'interprete legge il codice sorgente, lo analizza e lo esegue riga per riga o blocco per blocco. Questo processo consente agli sviluppatori di testare e modificare rapidamente il codice, poiché non è necessario attendere la compilazione completa del programma. Gli interpreti sono spesso utilizzati durante la fase di sviluppo e debugging, poiché offrono un feedback immediato sugli errori e sui risultati dell'esecuzione del codice.

Sebbene l'esecuzione concreta (o l'interpretazione classica) fornisca risultati esatti per una specifica esecuzione, non permette di dedurre proprietà generali del programma per tutti i possibili input. D'altra parte, il Teorema di Rice stabilisce che verificare proprietà semantiche non banali di un programma (come la terminazione o l'assenza di errori a runtime) è un problema indecidibile. Per cercare di affrontare questa limitazione, l'analisi statica moderna si affida a tecniche di approssimazione matematica.

Interpretazione Astratta

L'Interpretazione astratta, introdotta da Patrick e Radhia Cousot nel 1977, fornisce una teoria per l'approssimazione sicura (*sound*) delle strutture matematiche, applicata alla semantica formale dei programmi. Poiché non è possibile calcolare esattamente tutti gli stati infiniti di un programma, un interprete astratto sostituisce il dominio infinito dei valori concreti di esecuzione con un dominio semplificato e finito, chiamato **dominio astratto**. Questo dominio è tipicamente strutturato matematicamente come un *poset* (Partially Ordered

Set) o un reticolo (*Lattice*).

Il principio fondamentale di questa tecnica è la *conservativeness* (affidabilità) in cui l'analizzatore calcola uno stato approssimato ma matematicamente sicuro. L'interprete garantisce di non assumere mai proprietà false, pur accettando una perdita di precisione per mantenere l'analisi decidibile. Se durante l'analisi un'operazione produce incertezza o ambiguità matematica, il sistema degrada verso uno stato di totale ignoranza, comunemente rappresentato dal simbolo $\top$ (Top), evitando così di compiere deduzioni non sicure (*unsound*). Al contrario, lo stato di errore, codice irraggiungibile o variabile non inizializzata è rappresentato dal simbolo $\perp$ (Bottom).

L'astrazione si riflette anche nella valutazione del flusso di controllo. Quando l'interprete incontra diramazioni condizionali (If-Else), la guardia statica potrebbe valutarsi come $\top$ (incerta). In tali scenari, l'interprete esplora speculativamente entrambi i percorsi di esecuzione. Le memorie astratte risultanti dai due rami vengono poi riconciliate al punto di giunzione (*join-point*) avvalendosi dell'operatore matematico Least Upper Bound (LUB, $\sqcup$). Questo operatore calcola il minimo limite superiore nel reticolo, unificando gli stati discordanti.

La sfida maggiore nell'analisi statica è la valutazione dei cicli (While), poiché tentare di srotolarli all'infinito porterebbe all'Halting Problem. L'astrazione ridefinisce la dimensione dei possibili stati tramite l'utilizzo di domini astratti, ovvero domini finiti che forniscono una rappresentazione approssimata ma sicura degli stati del programma. I domini astratti, diversamente dai domini matematici concreti, invece di essere definiti da un insieme di valori univoci o di intervalli, sono definiti da proprietà dei valori stessi, ad esempio la parità o la disparità di un valore intero oppure, come mostrato all'interno di questo progetto, il segno.

L'interprete valuta iterativamente il corpo del ciclo, unendo continuamente lo stato della memoria successiva con quello precedente tramite l'operatore LUB ($M_{i+1} = M_i \sqcup \text{body}(M_i)$). Il calcolo termina quando l'iterazione non porta più ad alcun cambiamento, ovvero quando la memoria astratta raggiunge uno stato stazionario (il punto fisso). Poiché il dominio astratto (come il reticolo dei segni) ha un'altezza finita, la convergenza è matematicamente garantita, permettendo all'interprete di analizzare loop potenzialmente infiniti in pochi millisecondi.

Sintassi e semantica

Prima di poter analizzare o astrarre le proprietà semantiche di un programma, è strettamente necessario che la macchina sia in grado di interpretare il testo

grezzo scritto dal programmatore. Il codice sorgente non è intrinsecamente comprensibile per un analizzatore logico; deve essere prima sottoposto a un processo di analisi lessicale e sintattica. Questo processo trasforma il flusso di caratteri in una struttura dati gerarchica e navigabile, comunemente nota come albero sintattico, o *Parse Tree*.

Tradizionalmente, molti strumenti di analisi e compilazione eseguono ulteriori passaggi di trasformazione. Il Parse Tree viene spesso convertito in rappresentazioni intermedie più astratte e complesse, come l'Abstract Syntax Tree (AST), che rimuove i dettagli sintattici superflui, o il Control Flow Graph (CFG), che mappa esplicitamente tutti i possibili percorsi di esecuzione del programma. Sebbene queste strutture intermedie offrano vantaggi specifici per l'ottimizzazione del codice, esse introducono anche un notevole livello di complessità architetturale.

Per mantenere il sistema snello e focalizzato sull'interpretazione, il framework sviluppato in questo progetto adotta un approccio più diretto. L'architettura si affida ad ANTLR4 per l'analisi e il riconoscimento sintattico del codice sorgente. Tuttavia, il sistema bypassa deliberatamente la costruzione di rappresentazioni intermedie complesse, come l'AST o il CFG.

Al contrario, l'interprete valuta sia la semantica concreta che quella astratta operando direttamente sui nodi dell'albero sintattico (Parse Tree) generato originariamente da ANTLR4. Questo attraversamento logico viene gestito tramite l'implementazione del Visitor Pattern. L'utilizzo di questo pattern comportamentale permette all'analizzatore di "visitare" ogni nodo dell'albero e di applicarvi immediatamente le regole del sistema di tipi e dell'interpretazione astratta, garantendo un'elaborazione efficiente e una stretta aderenza tra la sintassi del linguaggio e la sua semantica formale.

Controllo dei tipi e validazione statica

In un linguaggio di programmazione, l'analisi semantica assume un significato compiuto solo se le operazioni logiche e aritmetiche vengono applicate a operandi compatibili. Il controllo dei tipi è fondamentale per identificare e scartare a priori i programmi mal formati, come il tentativo di sommare un intero con un valore booleano (ad esempio, l'espressione `5 + true`). Prima che avvengano le valutazioni semantiche o qualsiasi forma di astrazione del dominio, le proprietà di base devono essere imposte e verificate rigorosamente.

La validazione preventiva è solitamente affidata a un sistema di tipi statico (*Type System*). Questo componente analizza il codice (spesso attraversandone la rappresentazione sintattica) per assicurarsi che i tipi dichiarati siano coerenti all'interno di ogni singola espressione. In linguaggi strutturati, il sistema di tipi impone regole per distinguere tra tipi base (come gli scalari puri) e tipi complessi o di riferimento (come gli array), prevenendo a priori operazioni illegali come l'assegnazione accidentale di un intero a un puntatore di array.

La validazione dei tipi agisce a tutti gli effetti come una barriera di sicurezza propedeutica, comunemente definita *Type Guarding*. Le proprietà ben tipizzate del programma vengono garantite prima che il motore dell'interprete astratto prenda il controllo per calcolarne l'approssimazione. Qualora il sistema rilevi una violazione statica, come il tentativo di combinare stringhe e interi senza le opportune regole di conversione, l'analisi viene immediatamente bloccata.

In un'architettura di analisi statica ben progettata, questa barriera assicura che il successivo calcolo matematico (come la ricerca del punto fisso o l'applicazione dell'operatore limite superiore sui domini astratti) avvenga esclusivamente su un codice formalmente corretto, eliminando alla radice un'intera classe di errori di runtime prima ancora di astrarne il comportamento formale.

Indecidibilità e Halting Problem

Un analizzatore statico ideale dovrebbe essere in grado di determinare con assoluta certezza il comportamento a runtime di qualsiasi frammento di codice. Tuttavia, questo ambizioso obiettivo si scontra con limiti matematici invalicabili, primo fra tutti il celebre Problema della Fermata (*Halting Problem*) formalizzato da Alan Turing. Questo principio stabilisce l'impossibilità di creare un algoritmo universale in grado di determinare, per un programma arbitrario e un dato input, se la sua esecuzione terminerà o se proseguirà all'infinito.

A questa limitazione fondamentale si aggiunge una sua diretta generalizzazione, nota come Teorema di Rice. Tale teorema dimostra che qualsiasi proprietà semantica non banale di un programma è matematicamente indecidibile. In altre parole, non esiste alcun metodo automatico e generale capace di verificare in modo esatto proprietà come l'assenza di errori a runtime, il raggiungimento di stati specifici delle variabili o l'accessibilità di determinati blocchi di codice.

Nell'ambito dell'analisi statica, l'indecidibilità si manifesta nella sua forma più critica durante la valutazione dei cicli (come i costrutti *While*). Se un analizzatore tentasse di simulare l'esecuzione di un ciclo srotolandolo iterazione

dopo iterazione, rischierebbe di rimanere intrappolato in un'elaborazione senza fine, rendendo lo strumento inutilizzabile e cadendo vittima dello stesso problema della fermata che cerca di analizzare.

Poiché è matematicamente impossibile dedurre le proprietà semantiche di un programma calcolando il dominio infinito e concreto dei valori di esecuzione, l'analisi tradizionale non può essere applicata direttamente. Emerge quindi la necessità assoluta di ricorrere all'approssimazione matematica. Per analizzare staticamente i cicli senza incorrere in loop infiniti, l'indagine deve spostarsi su un piano logico differente: un piano in cui l'interprete esegue un calcolo iterativo (noto come *Fixpoint Computation* o calcolo del punto fisso) che garantisce di raggiungere uno stato stazionario, concludendo l'analisi in tempi rapidi e sicuri. Queste costrizioni teoriche costituiscono, di fatto, la giustificazione formale che rende necessaria l'adozione delle tecniche di interpretazione astratta.

Affidabilità e precisione

Dall'indecidibilità delle proprietà semantiche discende la necessità di analizzare il codice attraverso tecniche di approssimazione. Tuttavia, affinché uno strumento di analisi statica sia realmente utile in contesti ingegneristici e di sicurezza, l'approssimazione calcolata deve fornire garanzie matematiche rigorose. Questo principio introduce uno dei concetti cardine dell'Informatica Teorica: il compromesso tra *Soundness* (affidabilità) e precisione.

Un analizzatore statico ben progettato deve operare attraverso un'approssimazione sicura (*Sound Approximation*). Questo significa che il sistema calcola un'astrazione che garantisce di non mentire mai sui possibili stati del programma, evitando di compiere assunzioni false o non sicure (*unsound*). Se l'analizzatore dichiara che una variabile non assumerà mai valori negativi o che un errore è impossibile, tale affermazione deve essere inconfutabile per ogni possibile esecuzione.

Per mantenere questa infallibilità logica pur operando in un contesto indecidibile, l'analizzatore è costretto ad adottare un approccio rigorosamente conservativo (*Conservativeness*). Il principio di conservatività impone che il sistema preferisca perdere volontariamente una parte di precisione piuttosto che rischiare di trarre una conclusione errata o parziale. Ad esempio, di fronte a un'operazione matematicamente ambigua o a un aggiornamento di memoria in cui la destinazione esatta non è nota, l'analizzatore non tenta di indovinare il risultato. Al contrario, estende il ventaglio delle stime fondendo le opzioni possibili o degradando l'informazione verso uno stato di incertezza totale (spesso

denotato dal simbolo $\top$).

Questa espansione sicura del dominio permette di racchiudere tutti i possibili comportamenti del codice senza ometterne alcuno, mantenendo l'analisi decidibile. Il sacrificio mirato dell'esattezza in favore dell'affidabilità assoluta costituisce il paradigma fondamentale su cui si fonda l'Interpretazione Astratta, giustificando concettualmente l'uso di domini matematici semplificati per mappare gli stati di esecuzione.

Modellazione dei Programmi e Livelli di Analisi

Per eseguire un'analisi formale sul software, è innanzitutto necessario fissare un linguaggio di riferimento e modellare i programmi per poter ragionare matematicamente su di essi. L'analisi può operare su due livelli di astrazione distinti:

- **Analisi a livello di programma (Program-level analyses):** Vengono eseguite direttamente sul codice sorgente (es. scritto in C o Java) o sui file binari eseguibili. Questo approccio coinvolge tipicamente un *front-end* simile a quello di un compilatore, il quale si occupa di costruire gli alberi sintattici a partire dai file analizzati.
- **Analisi a livello di modello (Model-level analyses):** L'input non è il programma in un linguaggio reale, ma una descrizione formale che modella il programma stesso. Tali modelli necessitano di essere costruiti manualmente o calcolati tramite strumenti separati.

Nella modellazione, la struttura di controllo del programma viene spesso rappresentata tramite Grafi del Flusso di Controllo (Control Flow Graphs, CFG). In questa rappresentazione, i vertici corrispondono ai punti del programma, mentre gli archi indicano i passi della computazione e sono etichettati con l'azione corrispondente.

L'obiettivo dell'analisi è verificare il comportamento del software. Una proprietà semantica è definita in modo completo rispetto all'insieme delle esecuzioni di un programma. Una proprietà semantica si definisce "non banale" (*nontrivial*) quando esistono programmi che la soddisfano e programmi che non la soddisfano.

A causa del Teorema di Rice, la verifica automatica e perfetta di queste proprietà si scontra con limiti di computabilità insormontabili. Per poter procedere e rendere l'analisi computabile, è necessario accettare alcune limitazioni fondamentali: si deve rinunciare all'automazione totale oppure ci si deve focalizzare

solo su una classe ristretta di programmi.

Per modellare matematicamente le esecuzioni dei programmi si fa ricorso alla Semantica delle Tracce (*Trace semantics*). L'idea fondante è quella di costruire l'insieme di tutte le possibili computazioni di un programma partendo da un sistema di transizione, denotato come $\langle \Sigma, \tau \rangle$, definito sull'alfabeto degli stati del programma.

L'insieme di queste computazioni (sia finite che infinite) viene costruito come punto fisso di un trasformatore semantico. Durante il calcolo iterativo del punto fisso, ad ogni passo vengono conservate esclusivamente le tracce i cui stati iniziali appartengono alla relazione di transizione, eliminando progressivamente le tracce che non possono rappresentare computazioni valide del programma. Al limite di questo processo computazionale, si ottengono solamente le sequenze ammesse dal programma considerato.

Least Upper Bound (LUB, $\sqcup$)

Il **Least Upper Bound**, denotato con il simbolo $\sqcup$, è l'operatore matematico utilizzato per calcolare il limite superiore meno stringente tra due valori all'interno del reticolo. Il suo ruolo è essenziale durante la valutazione delle diramazioni condizionali (come i costrutti If-Else): poiché il valore di guardia potrebbe essere valutato come $\top$, l'interprete esplora entrambi i percorsi di esecuzione in modo speculativo. Al punto di giunzione, le memorie astratte finali derivanti dai due rami vengono riconciliate fondendole tramite il LUB:

$$\text{Memory}_{\text{final}} = \text{Memory}_{\text{Then}} \sqcup \text{Memory}_{\text{Else}}$$

Nel dominio astratto degli automi a stati finiti per l'analisi delle stringhe, l'applicazione dell'operatore LUB corrisponde direttamente al calcolo dell'unione matematica dei due automi.

Fixpoint Computation e Widening

La risoluzione dei cicli (*while*) è uno dei problemi centrali dell'analisi statica. Per aggirare il problema della fermata (*Halting Problem*), l'interprete esegue una **Fixpoint Computation** (calcolo del punto fisso). Il corpo del ciclo viene valutato iterativamente unendo il nuovo stato con quello precedente tramite l'operatore LUB:

$$M_{i+1} = M_i \sqcup \text{body}(M_i)$$

L'iterazione termina quando la memoria astratta raggiunge uno stato stazionario (il punto fisso), momento in cui ulteriori iterazioni non produrrebbero alcun cambiamento. Per domini con un'altezza finita, come il Reticolo dei Segni, la convergenza (spesso verso $\top$) è matematicamente garantita in pochi millisecondi.

Quando si utilizzano domini potenzialmente infiniti, come gli automi per le stringhe o le astrazioni per gli array, un ciclo potrebbe far crescere il valore astratto all'infinito. In questi casi, il calcolo del punto fisso introduce un operatore di **Widening**. Il widening forza una perdita volontaria di precisione (ad esempio, degradando l'automa a $\top$ se il numero dei suoi stati supera una certa soglia), garantendo la terminazione rapida e sicura dell'analisi.

Weak Updates

L'analizzatore statico deve garantire un'approssimazione sicura (*Sound Approximation*): non deve mai assumere proprietà false, pur accettando una perdita di precisione (conservatività). Questo principio è cruciale nella gestione di strutture dati mutabili e complesse, come l'astrazione degli Array in modalità *Index-Sensitive*. Quando l'indice a cui si tenta di accedere non è noto staticamente (viene valutato come $\top$), l'interprete non sa con certezza quale cella della memoria andrà a modificare. Invece di sovrascrivere un valore rischiando di compiere un'operazione errata (*strong update*), l'analizzatore è costretto ad applicare un **Weak Update** (aggiornamento debole). L'aggiornamento debole fonde (tramite l'operatore LUB, $\sqcup$) i nuovi dati in ingresso con i dati astratti già presenti, assicurando una stima conservativa che preserva la validità dell'analisi senza sovrascrivere indebitamente stati validi.

Implementazione

Specifiche

Dal punto di vista architetturale, il framework è interamente sviluppato in linguaggio Java ed elabora i file sorgente attraverso una rigorosa pipeline a più stadi. La prima fase prevede l'analisi sintattica del codice, affidata allo strumento ANTLR4, il quale si occupa di riconoscere la grammatica del linguaggio e di generare il relativo albero sintattico (Parse Tree). Una delle scelte progettuali più rilevanti del framework consiste nel bypassare deliberatamente la costruzione di rappresentazioni intermedie complesse come gli Abstract Syntax Tree (AST) o i Control Flow Graph (CFG). Al contrario, l'interprete valuta sia la semantica concreta che quella astratta operando in modo diretto sui nodi dell'albero sintattico avvalendosi del Visitor Pattern. Prima che avvenga l'interpretazione astratta vera e propria, un Type System statico attraversa l'albero per assicurare che la tipizzazione (ad esempio, interi e booleani) sia coerente. Questo processo, funge da barriera di sicurezza atta a scartare a priori i programmi mal formati, garantendo che le operazioni semantiche successive vengano applicate unicamente a costrutti validi.

Il nucleo dell'analizzatore è costituito dall'interprete astratto, il quale mappa lo stato delle variabili del programma all'interno di un dominio matematico finito, strutturato formalmente come un reticolo (Lattice). Il dominio di base utilizzato per le variabili numeriche è il Reticolo dei Segni, recentemente esteso per supportare un'analisi semantica estremamente granulare. Oltre agli stati canonici come BOTTOM (che indica errori o codice irraggiungibile), ZERO, POS (positivo), NEG (negativo) e TOP (incertezza totale), il sistema integra le astrazioni dello zero positivo ($0+$), dello zero negativo ($0-$) e del diverso da zero ($\neq 0$). Questa estensione risulta cruciale per gestire matematicamente le ambiguità: ad esempio, garantisce che l'operazione di divisione intera tra due numeri positivi restituisca formalmente l'astrazione $0+$, preservando così la soundness dell'analisi. La gestione del flusso di controllo impone sfide teoriche notevoli: in presenza di diramazioni condizionali (If-Else), l'interprete procede con un'esplorazione speculativa di entrambi i rami, le cui memorie astratte vengono infine riunificate al punto di giunzione sfruttando l'operatore matematico Least Upper Bound (LUB, $\sqcup$). Ancor più complessa è la valutazione dei cicli (While), che, se srotolati all'infinito, farebbero ricadere il sistema nel problema della fermata (Halting Problem). Per ovviare a questo limite, l'analizzatore esegue una Fixpoint Computation (calcolo del punto fisso), reiterando l'unione logica degli stati della memoria fino a raggiungere una condizione di stabilità. Basandosi sulla semantica delle tracce, questo calcolo iterativo conserva esclu-

sivamente le tracce che appartengono alla relazione di transizione, eliminando progressivamente quelle invalide e ottenendo, al limite, solo le sequenze effettivamente ammesse dal programma.

L'evoluzione del progetto prevede due espansioni fondamentali del linguaggio: l'introduzione delle stringhe e l'implementazione degli array. Per quanto concerne il dominio delle stringhe, il framework implementa una soluzione all'avanguardia basata sugli Automi a Stati Finiti (FSA), avvalendosi della libreria `dk.brics.automaton`. A differenza di un tradizionale reticolo piatto, che di fronte a stringhe discordanti degraderebbe rapidamente nello stato di incertezza totale (TOP) perdendo precisione formale, l'uso degli FSA permette di modellare il valore di una variabile come un vero e proprio linguaggio regolare. In questo dominio, l'operazione LUB utilizzata per riconciliare i rami condizionali corrisponde matematicamente al calcolo dell'unione degli automi. Inoltre, per garantire la rapida convergenza al punto fisso all'interno dei cicli, il sistema applica operatori di widening che limitano la complessità strutturale dell'automa, forzandone il rilassamento verso lo stato universale per evitare loop computazionali infiniti.

Parallelamente, l'integrazione degli array introduce la complessa problematica dell'indicizzazione dinamica all'interno dell'astrazione logica. L'architettura prevede la valutazione di due modelli teorici distinti. Il primo, noto come Smash Abstraction, semplifica drasticamente il problema collassando l'intera collezione di dati in un'unica variabile astratta riassuntiva; pur facilitando l'implementazione, questo metodo comporta la perdita totale delle informazioni posizionali degli elementi, applicando continuamente l'operatore LUB su ogni nuovo inserimento. Il secondo approccio, definito Index-Sensitive Abstraction, mira a preservare una mappatura rigorosa tra gli indici astratti e i rispettivi valori allocati. Tuttavia, quest'ultima metodologia richiede l'applicazione di complesse tecniche di weak update (aggiornamento debole). Quando l'indice di accesso a un array non è determinabile staticamente (risultando valutato come $\top$), l'analizzatore non possiede la certezza matematica relativa alla specifica cella di memoria soggetta a modifica. In osservanza del principio di conservatività (preferire la perdita di precisione all'assunzione di falsità logiche), l'interprete non sovrascrive il dato preesistente, ma lo fonde con il nuovo valore in ingresso tramite l'operatore LUB, assicurando che l'analisi rimanga formalmente sicura e affidabile in ogni contesto.

Guida rapida

Il framework sviluppato integra una suite di collaudo automatizzata, progettata per validare e dimostrare empiricamente i fondamenti teorici dell'Interpretazione Astratta illustrati nei capitoli precedenti. Questa sezione fornisce le direttive essenziali per la configurazione dell'ambiente, l'orchestrazione del processo di analisi e l'interpretazione dei risultati generati dal motore semantico.

Per garantire la corretta esecuzione del motore di analisi statica, l'ambiente di sviluppo deve soddisfare specifici requisiti software. È richiesta l'installazione del Java Development Kit (JDK) in versione 21 o superiore, sebbene l'intera architettura sia stata ottimizzata nativamente per l'esecuzione su JDK 24. La gestione del ciclo di vita del progetto, inclusa la risoluzione delle librerie esterne e le routine di build, è interamente affidata a Maven. La fase di compilazione standard tramite Maven si occupa inoltre di innescare automaticamente la generazione del parser ANTLR4 a partire dalla grammatica definita, un passaggio propedeutico per preparare il framework alla costruzione degli alberi sintattici.

L'architettura include un test bench automatizzato che agisce come punto di ingresso principale per l'analizzatore statico. Durante l'esecuzione del runner principale, il sistema esegue una scansione autonoma della directory `examples/`, la quale funge da contenitore per tutti i file di codice sorgente redatti nel linguaggio di test supportato. Per ogni caso di test individuato, la pipeline procedurale opera sequenzialmente: il framework stampa innanzitutto a video il codice sorgente originale, procede con l'esecuzione della validazione statica e, infine, calcola la semantica astratta (i segni) per le variabili coinvolte nel programma.

L'output generato dalla suite di collaudo non si limita alla visualizzazione del valore finale assunto dalle variabili, ma è strutturato per dimostrare sul campo le garanzie matematiche intrinseche dell'analizzatore:

- **Type Guarding (Validazione dei Tipi):** I log di esecuzione confermano che le proprietà sintattiche e semantiche ben tipizzate vengono rigorosamente imposte e verificate prima che le funzioni di trasferimento logico sui domini astratti prendano il controllo dell'analisi.
- **Diramazioni Precise (Precise Branches):** Quando l'analizzatore esplora percorsi di esecuzione sovrapposti (come nel caso dei rami logici di un blocco condizionale) e questi concordano sul medesimo segno di output, il sistema mantiene una precisione esatta senza inutili perdite di informazione (ad esempio, preservando e restituendo l'astrazione strettamente positiva, +).

- **Conservatività (Conservativeness):** Di fronte a operazioni aritmeticamente ambigue, come la sottrazione tra due astrazioni positive, il sistema adotta un approccio rigorosamente conservativo degradando in modo sicuro verso lo stato di incertezza totale ($\top$), piuttosto che rischiare di formulare un'assunzione non sicura o errata.
- **Terminazione dei Cicli (Loop Halting):** L'analisi dei costrutti iterativi, quali i cicli che incrementano variabili fino a raggiungere un limite costante, innescano correttamente il motore di calcolo del punto fisso. L'output dimostra come il sistema converga in modo stabile verso $\top$ in frazioni di millisecondo, garantendo la terminazione dell'analisi e neutralizzando il rischio di ricadere in loop infiniti.

Prendendo come modello strutturale il layout illustrato nel file "Screenshot 2026-09-01 alle 11.36.37.png", questa sezione fornisce le direttive operative essenziali per la configurazione dell'ambiente, la compilazione e l'esecuzione del framework di analisi statica.

Requisiti

Per garantire la corretta esecuzione del motore semantico, l'ambiente di sviluppo deve soddisfare i seguenti requisiti software:

- **Java Development Kit (JDK):** È richiesta l'installazione della versione 21 o superiore, sebbene l'intera architettura sia stata ottimizzata nativamente per l'esecuzione su JDK 24.
- **Maven:** Necessario per la gestione delle dipendenze esterne e il ciclo di build del progetto tramite il file `pom.xml`.
- **ANTLR4:** Il plugin per Maven (configurato all'interno del `pom.xml`) è richiesto per la generazione automatica delle classi del parser e del lexer a partire dal file di grammatica.

Compilazione del progetto

La fase di compilazione tramite Maven si occupa di innescare automaticamente la generazione del parser ANTLR4, un passaggio propedeutico indispensabile per permettere al framework di costruire gli alberi sintattici. Dalla directory radice del progetto, dove risiede il file `pom.xml`, è necessario eseguire il seguente comando da terminale per generare i sorgenti e compilare le classi Java:

```
mvn clean compile
```

Eseguire un programma e Test Bench

L'architettura del progetto include un *test bench* automatizzato che agisce come punto di ingresso per l'analizzatore statico. Per avviare l'interprete, sempre dalla directory radice del progetto, è possibile eseguire il runner principale tramite il seguente comando Maven:

```
1 mvn exec:java -Dexec.mainClass="it.univr.pl.  
    MainAbstractInterpreter"
```

Una volta avviata, l'applicazione eseguirà una scansione autonoma della directory `examples/`, la quale funge da contenitore per tutti i file di codice sorgente redatti nel linguaggio supportato dall'interprete. Per ogni file analizzato, il sistema stamperà a video il codice sorgente originale, avvierà il controllo dei tipi e, infine, eseguirà la valutazione astratta calcolando la semantica dei segni per le variabili del programma.

Sintassi

L'esecuzione di un'analisi a livello di programma richiede tipicamente un *front-end*, simile a quello di un compilatore, incaricato di elaborare il codice sorgente per costruire i relativi alberi sintattici. Per poter eseguire ragionamenti formali, è innanzitutto necessario fissare la sintassi del linguaggio di programmazione che si intende analizzare. Il progetto fa riferimento a un linguaggio imperativo semplice, il quale supporta nativamente valori scalari, variabili di programma, operatori binari (come addizione e sottrazione) e operatori di confronto (come $<$, $\leq$ e $==$).

Il processo di riconoscimento sintattico all'interno del framework è interamente gestito tramite ANTLR4. La grammatica formale del linguaggio è specificata all'interno del file dedicato `Abs.g4`. Tale grammatica definisce le regole di derivazione per le espressioni del linguaggio, includendo il supporto per operatori specifici come il meno unario (`SUB exp`), il riconoscimento dei token per le stringhe letterali e l'utilizzo delle parentesi quadre per la dichiarazione e l'indicizzazione dinamica degli array.

Mentre i tradizionali modelli di analisi ricorrono spesso alla costruzione di rappresentazioni intermedie, come i Grafi del Flusso di Controllo (CFG), per modellare la struttura di controllo del programma, l'architettura di questo framework adotta un approccio più diretto. Il sistema, infatti, bypassa deliberatamente la generazione di strutture intermedie complesse come gli *Abstract Syntax Tree* (AST) o i CFG. Al contrario, la valutazione della semantica, sia concreta che astratta, avviene operando direttamente sui nodi dell'albero sintattico (*Parse Tree*) generato da ANTLR4, avvalendosi dell'implementazione del *Visitor Pattern*.

Interpretazione Astratta

Introdotta da Patrick e Radhia Cousot nel 1977, l'Interpretazione Astratta fornisce una teoria per l'approssimazione sicura (*sound approximation*) delle strutture matematiche, applicata principalmente alla semantica formale dei programmi. Poiché verificare proprietà semantiche non banali è indecidibile (Teorema di Rice), un interprete astratto sostituisce il dominio infinito dei valori concreti di esecuzione con un **Dominio Astratto** finito, strutturato come reticolo (*Lattice*).

Soundness e Approssimazione

L'analizzatore calcola uno stato approssimato ma sicuro: garantisce di non assumere mai proprietà false, pur potendo perdere precisione per mantenere la

decidibilità. In presenza di incertezza, il sistema degrada verso uno stato di totale ignoranza ($\top$).

LUB e Convergenza (Fixpoint)

- **Diramazioni Condizionali (If-Else):** Poiché la guardia statica potrebbe valutarsi come $\top$, l'interprete esplora speculativamente entrambi i rami. Le memorie astratte finali vengono riconciliate al *join-point* usando l'operatore matematico **Least Upper Bound (LUB, $\sqcup$)**:

$$\text{Memory}_{\text{final}} = \text{Memory}_{\text{Then}} \sqcup \text{Memory}_{\text{Else}}$$

- **Cicli (While):** Per risolvere staticamente il problema della fermata (*Halting Problem*), l'analisi dei loop esegue una **Fixpoint Computation** (calcolo del punto fisso). L'interprete valuta iterativamente il corpo del ciclo finché la memoria astratta non raggiunge uno stato stazionario:

$$M_{i+1} = M_i \sqcup \text{body}(M_i)$$

Grazie all'altezza finita del reticolo dei segni, la convergenza in $\top$ è matematicamente garantita in pochi millisecondi senza il rischio di loop infiniti.

Il Dominio dei Segni (Sign Lattice)

Il dominio concreto degli interi $\mathbb{Z}$ viene astratto nel Dominio dei Segni. Nel progetto, il reticolo base è stato esteso per supportare un'analisi più granulare e precisa, in preparazione all'analisi dei linguaggi dinamici.

Struttura del Reticolo Esteso

Il reticolo attuale, definito dalla classe `SignType`, supporta le seguenti astrazioni:

- $\perp$ (**Bottom**): Rappresenta uno stato non inizializzato, codice irraggiungibile o un errore statico a runtime (es. divisione per zero).
- **NEG** ($-$): Numeri strettamente negativi.
- **ZERO** (0): Esattamente zero.
- **POS** ($+$): Numeri strettamente positivi.
- **ZERO_PLUS** ($0+$): Numeri maggiori o uguali a zero (zero o positivi).
- **ZERO_MINUS** ($0-$): Numeri minori o uguali a zero (zero o negativi).
- **NOT_ZERO** ($\neq 0$): Numeri positivi o negativi, ma certamente non zero. Generato ad esempio dal LUB tra **POS** e **NEG**.
- $\top$ (**Top**): Incertezza totale (potrebbe essere qualsiasi intero).

Operazioni Aritmetiche Astratte

La classe `SignValue` è responsabile della gestione delle operazioni aritmetiche tra gli elementi del reticolo. Tali operazioni garantiscono l'affidabilità (*soundness*) dell'analisi:

- **Addizione** (`add`): Mantiene la precisione se i segni concordano (es. $\text{POS} + \text{POS} = \text{POS}$, $\text{POS} + 0+ = \text{POS}$). Se si sommano valori opposti o misti che coprono l'intero asse numerico (es. $\text{POS} + \text{NEG}$ oppure $\text{NOT_ZERO} + 0+$), il risultato degrada in $\top$.
- **Sottrazione** (`sub`): L'operatore meno unario inverte il segno (es. $\text{POS} \rightarrow \text{NEG}$), permettendo di calcolare la sottrazione tramite l'addizione con l'operando invertito.

- **Moltiplicazione** (`mul`): Calcola accuratamente i prodotti combinati, restituendo valori come $0+$ o $0-$ quando si moltiplicano segni incerti (che includono lo zero) con segni certi.
- **Divisione** (`div`): Gestisce in modo conservativo il troncamento della divisione intera in Java. Ad esempio, la divisione tra POS e POS restituisce $0+$, considerando il caso in cui il numeratore sia minore del denominatore (es. $1/2 = 0$). Se vi è certezza di dividere per zero, viene restituito lo stato di errore $\perp$.

Componenti dell'inteprete